Radar Operacional

Descrição do projeto, funcionamento e estado atual

Versão de 07/10/2026 - etapa 2 (motor local com canal Teams)

1. O que é o projeto

O Radar Operacional é um sistema em Python que roda na máquina do servidor e entrega, de forma automática, informações operacionais para as pessoas certas. Em vez de a pessoa precisar abrir o Power BI e procurar o que mudou, o sistema consulta os dados, descobre o que é novo e envia um aviso no Teams (e, no futuro, por e-mail e SharePoint).

A ideia resumida é esta: o BI responde "como está?"; o Radar responde "o que mudou e o que precisa da minha atenção agora?". Ele não substitui o BI nem o controle oficial: é um aviso auxiliar.

A primeira aplicação é o aviso de **novas CDs de não conformidade do dia** (classificação Escape, centro SJK1, CTs da GFW), lidas direto do BigQuery.

2. Como ele funciona, passo a passo

Cada envio é uma **tarefa** descrita em um arquivo YAML. Quando uma tarefa roda, o motor executa sempre a mesma sequência:

1. **Consulta.** A fonte (BigQuery) executa o SQL da tarefa e devolve as linhas. O SQL fica em um arquivo separado e é executado exatamente como foi escrito.
2. **Validação.** O motor confere se as colunas esperadas existem. Se a tabela de origem mudar de estrutura, a execução falha com o código SCHEMA_MUDOU em vez de enviar algo errado.
3. **Identificação.** A chave de cada item (o número da Nota) é transformada em um código HMAC (índice cego). O banco só guarda esse código, nunca o número da nota em claro.
4. **O que é novo?** O motor compara os códigos com os já avisados. Itens repetidos na mesma leitura e itens já avisados antes são descartados.
5. **Mensagem.** Com os itens novos, monta uma mensagem única: título, "dados de HH:MM", contagens, tabela de itens e o aviso de que não substitui o SAP.
6. **Prévia.** Gera um HTML local para você ver exatamente o que seria enviado. Uma cópia cifrada da mensagem também fica no banco.
7. **Entrega.** Somente em execução real e somente se houver item novo, a mensagem vai para os canais externos permitidos (hoje, o Teams por webhook). Sem novidade, nenhuma mensagem é enviada.
8. **Estado.** Só depois de a entrega dar certo o motor grava o snapshot e marca os itens como avisados. Se o envio falhar, nada é marcado e a próxima execução

tenta de novo.

9. **Registro.** Cada execução entra no histórico com status, contagens e, em caso de falha, apenas um código curto de erro.

O resultado prático é que rodar o mesmo comando duas vezes não duplica avisos, e uma falha de rede não faz nenhuma CD ser perdida.

3. Modos de execução

Modo	O que faz	Grava estado?	Envia ao Teams?
Dry-run (padrão)	Consulta, gera a prévia e a cópia cifrada da mensagem.	Não	Não
Real (--real)	Faz tudo do dry-run, entrega e marca como avisado.	Sim, após entregar	Sim, se permitido pelo sandbox
Todos (--todos)	Mostra todos os itens da leitura, ignorando o que já foi avisado.	Conforme o modo	Conforme o modo

Uma tarefa nasce com **ativa: false**. Para registrar uma execução real de uma tarefa inativa é preciso usar --forcar-inativa de propósito.

4. Arquitetura e módulos

Pasta / arquivo	Responsabilidade
radar/sources/	Fontes de dados. bigquery.py executa o SQL e estima o custo (dry_run); demo.py lê dados fictícios para testes.
radar/engine/runner.py	Orquestra a execução: consulta, validação, índice cego, novidades, mensagem, entrega, estado e log.
radar/channels/	Canais de entrega. preview.py gera o HTML local; teams_webhook.py monta o cartão e posta no Teams; base.py define o erro de envio.
radar/storage/	crypto.py (cifragem Fernet e índice cego HMAC) e db.py (SQLite, o único lugar que fala SQL).
radar/segredos.py	Leitura e gravação de segredos, como a URL do webhook, no Credential Manager.
radar/config.py e __main__.py	Carregam configurações e tarefas; oferecem a linha de comando.
sql/	Consultas. cds_novas_do_dia.sql é a consulta original, sem alterações.
tasks/	Uma tarefa por arquivo YAML (cds_novas_do_dia, demo, demo_teams).

Pasta / arquivo	Responsabilidade
config/settings.yaml	Projeto do BigQuery, caminhos, retenção e sandbox.
tests/	29 testes automatizados, todos com dados fictícios.

A regra de desenho é que cada canal e cada fonte seguem uma interface simples. Para criar um novo canal (e-mail, SharePoint) ou uma nova fonte, basta escrever um módulo novo, sem mexer no motor.

5. A tarefa atual: novas CDs do dia

A tarefa cds_novas_do_dia usa a consulta que você definiu, mantida exatamente como está. Ela filtra CDs do centro SJK1, tipo CD, das cinco CTs da GFW, com classificação contendo "ESCAPE" e status I0070 (aberta) ou I0072 (encerrada). A última medida de cada nota é escolhida por deduplicação, e o resultado traz apenas as CDs do **dia mais recente** da base.

O envio mostra: Nota, NS do avião, CT atual, tipo de defeito, item de codificação e descrição do problema. A chave de controle de repetição é a Nota.

Na validação com dry-run real no BigQuery, a consulta leu cerca de 3,9 GB e o resultado conferiu com o Power BI.

6. Segurança e tratamento dos dados

- **Cifragem por campo.** O conteúdo das mensagens e dos itens é cifrado com Fernet (AES + HMAC) antes de ir para o SQLite. Em claro ficam apenas ids internos, datas, status, contagens e códigos de erro.
- **Índice cego.** A comparação de "já avisei?" usa HMAC-SHA256 com chave própria, diferente da chave de cifragem. O banco compara códigos sem nunca ver o valor. Um teste confirma que nem o texto das CDs nem a Nota aparecem em claro no arquivo do banco.
- **Chaves.** Ficam no Credential Manager do Windows (ou em variável de ambiente, só em desenvolvimento). Nunca no código, no banco ou no repositório. O MultiFernet permite trocar a chave sem perder dados antigos. O backup das chaves deve ser separado do banco: sem elas, o histórico é irrecuperável.
- **Segredos de canal.** A URL do webhook do Teams é guardada com o comando guardar-segredo e nunca aparece em log, mensagem de erro ou traceback.
- **Logs.** Só códigos e tipos de erro (por exemplo TEAMS_HTTP_401), nunca conteúdo.
- **Sandbox.** Enquanto ativo, só os canais listados em canais_permitidos recebem envios reais. A tarefa das CDs reais está configurada apenas com a prévia local.
- **Previews em disco.** O HTML contém dado operacional em claro. Ficam em uma pasta temporária e o comando limpar apaga o que passar de 24 horas. Esta é a parte que a cifra por campo não cobre, e é um dos motivos para confirmar o

BitLocker com a TI.

7. Canal Teams (webhook do Workflows)

O Python monta um Adaptive Card e faz um POST na URL de um workflow criado no Teams com o modelo "Send webhook alerts to a channel". O workflow apenas publica o que recebe; toda a lógica fica no código.

- O cartão mostra até 10 itens, corta textos longos e reduz a quantidade de itens se passar de cerca de 24 KB (o Teams recusa cartões acima de aproximadamente 28 KB), avisando "+N não exibidos".
- Erros 400, 401, 403 e 404 não são repetidos (o problema é de configuração). Erros 429 e 5xx têm uma segunda tentativa.
- Só URLs https são aceitas (localhost é permitido apenas para testes).
- Se o envio falhar, o erro vira um código curto e nada é marcado como avisado.

8. Comandos disponíveis

Comando	Função
`python -m radar init-keys`	Gera e guarda as chaves de cifragem e de HMAC.
`python -m radar guardar-segredo NOME`	Guarda um segredo (a URL do webhook) sem mostrar na tela.
`python -m radar teams-teste teste`	Envia uma mensagem de teste, sem dado operacional.
`python -m radar listar`	Lista as tarefas.
`python -m radar custo TAREFA`	Estima os bytes que a consulta leria no BigQuery.
`python -m radar run TAREFA`	Dry-run: gera a prévia e não grava nem envia nada.
`python -m radar run TAREFA --real`	Execução real: entrega e marca como avisado.
`python -m radar historico`	Mostra as últimas execuções.
`python -m radar limpar`	Aplica a retenção de mensagens e previews.
`python -m pytest`	Roda os 29 testes.

9. Estado atual

Pronto e validado:

- Motor completo (consulta, validação, índice cego, novidades, mensagem, entrega, estado, log).

- Armazenamento SQLite com cifragem por campo e índice cego.
- Fonte BigQuery: autenticada com a sua conta, dry-run de custo funcionando e execução real conferida com o Power BI.
- Canal de prévia em HTML.
- Canal Teams implementado e coberto por testes automatizados contra um servidor local falso.
- 29 testes automatizados passando.

Implementado, mas ainda não validado no ambiente real:

- Envio ao Teams de verdade. A criação do webhook na equipe de testes estava em andamento; falta rodar teams-teste e demo_teams e confirmar que o cartão chega bem.
- Execução sem supervisão: o login do gcloud está no seu usuário do Windows, e o agendamento precisa rodar com esse mesmo usuário.

Ainda não existe:

- Agendamento no Task Scheduler e alerta de que o robô não rodou (heartbeat).
- Interface web local para gerenciar tarefas.
- Perfis e preferências por pessoa.
- Canais de e-mail e SharePoint.
- Consulta de estoque de CDs abertas, necessária para "o que mudou?" e "parada há X dias".

10. Limitações e cuidados conhecidos

- **Consulta do dia mais recente.** Ela serve para "novas do dia", mas não para diff de estoque nem para alertas de CD parada. Se um dia não tiver CDs novas, a "data mais recente" passa a ser a de uma CD antiga; a anti-repetição evita reenviar o que já foi avisado, mas a mensagem desse dia não é uma novidade real.
- **Status não aparece no envio.** A consulta inclui I0070 (aberta) e I0072 (encerrada), mas não devolve a coluna de status, então a mensagem não diferencia as duas. Como a consulta foi mantida exatamente como está, isso fica como observação.
- **Defasagem dos dados.** O aviso nunca será mais atualizado que a carga SAP para o BigQuery; por isso a mensagem sempre mostra "dados de HH:MM".
- **Mais de um canal externo.** Se uma tarefa tiver dois canais e só o segundo falhar, o primeiro será reenviado na nova tentativa. Com um canal só, isso não ocorre.
- **Dependência da sua conta.** O login do BigQuery e o dono do workflow do Teams são você. Se a conta for desativada, o robô para. Para produção, o ideal é conta de serviço e um co-proprietário do workflow.
- **Um PC não é servidor.** Atualizações, reinício e queda de rede podem parar tudo; por isso o heartbeat é prioridade.

- **Dado de aviação.** O que pode circular por Teams e e-mail precisa ser confirmado com a TI e a segurança da informação antes de enviar CDs reais.

11. Próximos passos sugeridos

1. Concluir o teste do Teams com dados fictícios (guardar-segredo, teams-teste, demo_teams).
2. Agendar no Task Scheduler, sem login interativo, com heartbeat e log, e acompanhar alguns dias.
3. Definir com a TI o que pode ir para o Teams e liberar o canal para a tarefa real, ainda no canal de teste.
4. Criar a consulta de estoque de CDs abertas e passar a usar snapshots para o "o que mudou?".
5. Interface web local (somente em 127.0.0.1) para listar tarefas, rodar agora, ver prévias e histórico.
6. Perfis e preferências por pessoa, usando o resultado do Forms piloto.
7. E-mail e SharePoint, quando as permissões do app registration estiverem confirmadas.

12. Pendências com a TI

- Confirmar BitLocker no servidor (cobre temporários e previews).
- Conta de serviço para o BigQuery, para o robô não depender do seu login.
- Permissões do app registration (Mail.Send e Sites.Selected) e política de webhooks do Teams.
- Classificação da informação: o que pode circular por Teams, e-mail e SharePoint.
- Co-proprietário para o workflow do Teams.